

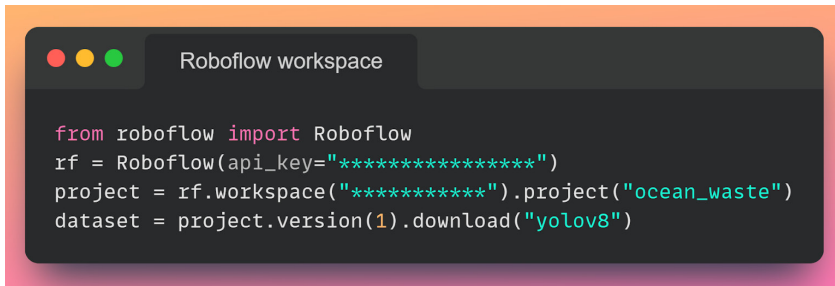
Step 2: Checking the proper installations for Ultralytics

Figure 3 shows the same. If there is any error found here then we should uninstall the library and reinstall it.

Step 3: Importing the desired data set from the Roboflow workspace directly using API keys

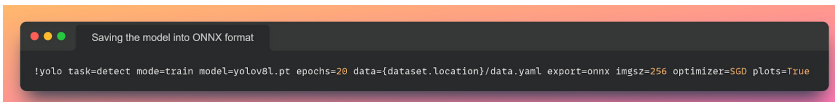
YOLOv8 is the latest version of YOLO by Ultralytics. As a cutting-edge, state-of-the-art model,

YOLOv8 builds on the success of previous versions, introducing new features and improvements for enhanced performance, flexibility, and efficiency. It supports a full range of vision AI tasks, including detection, segmentation, pose estimation, tracking, and classification. This versatility allows users to leverage YOLOv8's capabilities across diverse applications and domains. This set of codes helps in getting a data set from Roboflow directly in the form of code, and unzip the data and its annotations to the required format of YOLOv8. Figure 4 shows the snippet for the same.



```
from roboflow import Roboflow
rf = Roboflow(api_key="*****")
project = rf.workspace("*****").project("ocean_waste")
dataset = project.version(1).download("yolov8")
```

Figure 4: Import data set from Roboflow, and unzip the data and its annotations



```
!yolo task=detect mode=train model=yolov8l.pt epochs=20 data={dataset.location}/data.yaml export=onnx imgsz=256 optimizer=SGD plots=True
```

Figure 5: Save the model with the parameters listed into the ONNX format

Note: Beware of sharing this API key publicly. It is a private key linked to your Roboflow account.



Figure 6: Result image



Figure 7: The model is detecting the plastic bags with 80% accuracy

Step 4: Saving the model into ONNX format

Figure 5 shows the various parameters stated. This is a CLI command for training the YOLOv8 model.

The parameters used here are:

1. Task = Detect – Indicates the detection task
2. Mode = Train – Indicates the training task
3. Model = YOLOv8l.pt – Indicates which model to use
4. Data = {dataset.location}/yaml – Indicates location of data and YAML file
5. Epochs = 20 -- Indicates number of iterations
6. Imgsz = 256 -- Indicates the size of image in training
7. Plots = True -- Indicates display plots after running all the epochs
8. Optimizer = SGD – Indicates using stochastic gradient descent
9. Export = ONNX – Indicates saving the model in ONNX format

Results

All the results are stored in the `runs/detect/train` folder path. To use this